

UNIVERSITÉ DE SHERBROOKE · ÉCOLE DE GESTION

GIS805

Session 09 / 12

Les quatre types de tables de faits : transaction, snapshot, accumulating et factless

Structuration et analyse multidimensionnelle

C

PROFESSEUR

**Prof. Carlos Denner
dos Santos**

Département des systèmes
d'information et méthodes
quantit...

DATE

11 juin 2026

LIEU

Longueuil

HORAIRE

19 h 00 – 22 h 00

Votre séance, en bref

Voici comment se déroule votre soirée — 3 temps forts.

Jour 1 (~60 min)

Théorie : 4 types de faits + carte NexaMart

Jour 2 (~70 min)

Exercices 1 et 2 : transaction + snapshot

Jour 3 (~70 min)

Exercices 3 et 4 : accumulating + factless

Comment utiliser ce guide cette semaine

Séance 9 — travail en autonomie, pas de cours magistral

- Pas de séance en classe cette semaine — vous avancez à votre rythme
- **Durée estimée** : 3 h 30 réparties sur 3 à 4 jours
- **Jour 1 (jeudi)** : regarder la vidéo NotebookLM (voir diapositive suivante), lire les diapositives de théorie, puis `make generate` & `make load`
- **Jour 2 (vendredi)** : Exercices 1 et 2 (transaction + snapshot)
- **Jour 3 (samedi)** : Exercices 3 et 4 (accumulating + factless)
- **Jour 4 (dimanche)** : Brief exécutif, arbre de décision, git push

i Le point de départ est la vidéo de synthèse NotebookLM — voir la diapositive suivante.

Comment utiliser ce guide cette semaine (suite)

- ⚠ **Échéance** : avant le 15 juin 18h00 — la séance S10 est le Comité du board (non noté)

Commencez ici — la vidéo de synthèse NotebookLM

Étape 0 : regardez avant de toucher au code (~6 min)

- J'ai préparé une vidéo de synthèse sur NotebookLM qui remplace le cours magistral de cette semaine
- **Lien vidéo :** [Regarder la vidéo de synthèse S09 — 6 min](#)
- Elle couvre la théorie des 4 types de faits, les pièges classiques et le mapping NexaMart
- Le même lien est dans le guide de lab (section 0) si vous le perdez
- **Ensuite seulement :** lisez les diapositives de théorie, puis `make generate` & `make load`

 Ce lien pointe vers le notebook que j'ai créé pour vous — rien à téléverser, regardez directement.



*Quels processus NexaMart sont
transactionnels, quels sont des
snapshots, et quels sont de simples
presences ?*

— Le CEO de NexaMart Group, séance 9

Pas toutes les tables de faits se ressemblent

4 types fondamentaux pour 4 types de questions

- **Jusqu'ici** : fact_sales est une table de transactions
- Mais l'inventaire quotidien n'est pas une transaction
- Le pipeline de commandes n'est pas un événement ponctuel
- L'exposition promotionnelle n'a pas de mesure numérique
- Kimball définit 4 types de tables de faits pour ces 4 situations

01 Type 1 : Transaction

Un événement ponctuel, une ligne

La table de faits transactionnelle

Le type le plus courant -- une ligne par événement

- Chaque ligne = un événement qui s'est produit une fois (vente, retour, paiement)
- **Grain typique** : une ligne de commande a une date
- **Mesures** : revenue, quantite, cout, discount
- Additif sur toutes les dimensions
- **Exemple NexaMart** : fact_sales, fact_returns

DDL : table de faits transactionnelle

fact_orders_transaction -- une ligne = une transaction

```
-- GRAIN : une ligne = une transaction atomique (transaction_id)
--          transaction_type ∈ {sale, return, exchange}
-- INSERT-ONLY : on n'update jamais une transaction passée
CREATE OR REPLACE TABLE fact_orders_transaction AS
SELECT
    f.transaction_id,
    CAST(f.transaction_date AS DATE) AS transaction_date,
    f.transaction_type,
    p.product_key,
    st.store_key,
    c.customer_key,
    CAST(f.quantity AS INTEGER) AS quantity,
    CAST(f.amount AS DECIMAL(12,2)) AS amount
FROM raw_fact_orders_transaction f
JOIN dim_product p ON p.product_id = f.product_id
JOIN dim_store st ON st.store_id = f.store_id
JOIN dim_customer c ON c.customer_id = f.customer_id
                    AND CAST(f.transaction_date AS DATE)
                        BETWEEN c.effective_from AND c.effective_to;

-- Vérification attendue :
-- SELECT COUNT(*), COUNT(DISTINCT transaction_id) FROM fact_orders_transaction; -- égaux
-- SELECT SUM(amount) FROM fact_orders_transaction; -- total additif
```

❗ Vérification : `SELECT COUNT(*), SUM(amount) FROM fact_orders_transaction` — comparez avec `raw_fact_orders_transaction`. Le grain est `transaction_id` (total = distinct). `fact_orders_transaction` expose `amount/quantity`, pas `line_total/cost`.

02

Type 2 : Periodic Snapshot

Une photo a un moment donne

Le snapshot periodique

L'état a un instant, pas un événement

- Chaque ligne = l'état d'une entite a une date donnée
- **Grain typique** : un produit dans un magasin a une date
- **Mesures** : quantite_en_stock, valeur_stock, jours_stock
- **Semi-additif** : le stock ne s'additionne PAS sur le temps
- **Exemple NexaMart** : fact_inventory_snapshot

❗ Erreur classique : calculer le stock a partir des transactions. Un snapshot PREND une photo -- il ne calcule rien.

DDL : snapshot periodique

fact_daily_inventory -- une photo par produit × magasin × jour

```
-- GRAIN : un produit × un magasin × un jour
-- INSERT-ONLY (nouvelle photo chaque jour, l'ancienne est conservée)
CREATE OR REPLACE TABLE fact_daily_inventory AS
SELECT
    ri.snapshot_id,
    CAST(ri.snapshot_date AS DATE)           AS snapshot_date,
    p.product_key,
    st.store_key,
    CAST(ri.quantity_on_hand AS INTEGER)     AS quantity_on_hand,
    CAST(ri.quantity_on_order AS INTEGER)    AS quantity_on_order,
    CAST(ri.days_of_supply AS DECIMAL(5,1))  AS days_of_supply
FROM raw_fact_daily_inventory ri
JOIN dim_product p  ON p.product_id = ri.product_id
JOIN dim_store   st ON st.store_id  = ri.store_id;

-- Vérification attendue :
-- SELECT COUNT(*) FROM fact_daily_inventory; -- ~6 500 lignes (produits × magasins × jours)
-- SELECT AVG(quantity_on_hand) FROM fact_daily_inventory; -- stock moyen
-- PAS de SUM(quantity_on_hand) sur le temps !
```

❗ Vérification : le grain est product_key × store_key × snapshot_date. Un SELECT DISTINCT sur ces 3 colonnes doit retourner le même COUNT que SELECT *.

Semi-additivite : le piege du snapshot

ADDITIF (OK PARTOUT)

- SUM(revenue) par région : OK
- SUM(revenue) par mois : OK
- SUM(revenue) par région par mois : OK
- Fonctionne sur TOUTES les dimensions

SEMI-ADDITIF (DANGER SUR LE TEMPS)

- SUM(stock) par magasin : OK
- SUM(stock) par mois : FAUX !
- 100 unites lundi + 100 mardi != 200 unites
- Utilisez AVG() ou LAST() sur la dimension temps

SUM vs AVG : la démonstration par l'exemple

Exécutez ces deux requêtes sur vos données et comparez

```
-- ❌ FAUX : SUM sur le temps (gonfle le stock)
SELECT
    p.category,
    SUM(i.quantity_on_hand) AS stock_cumule -- absurde : additionne des photos
FROM fact_daily_inventory i
JOIN dim_product p ON p.product_key = i.product_key
GROUP BY 1
ORDER BY 2 DESC;

-- ✓ CORRECT : AVG sur le temps (stock moyen par catégorie)
SELECT
    p.category,
    ROUND(AVG(i.quantity_on_hand), 1) AS stock_moyen,
    MAX(i.quantity_on_hand)          AS stock_max,
    MIN(i.quantity_on_hand)          AS stock_min
FROM fact_daily_inventory i
JOIN dim_product p ON p.product_key = i.product_key
GROUP BY 1
ORDER BY 2 DESC;
```

❗ Résultat attendu : le SUM sera 10-30× plus grand que le AVG × nb_jours. Notez l'écart dans votre brief.

Transaction vs Periodic Snapshot

TRANSACTION

- Un événement ponctuel
- On insere, on ne modifie jamais
- Additif sur toutes les dimensions
- Grain : un événement a un moment

PERIODIC SNAPSHOT

- Un état a un moment
- Nouvelle photo a chaque periode
- Semi-additif (pas sur le temps)
- Grain : une entite a une date

03 Type 3 : Accumulating Snapshot

Le cycle de vie complet d'un processus

Le snapshot accumulant

Une ligne par processus, mise à jour à chaque étape

- **Le pipeline de commandes NexaMart** : order > payment > pick > ship > deliver
- Une ligne par commande, avec une colonne date par étape
- **Au debut** : seule order_date est remplie, les autres sont NULL
- **A chaque étape** : on UPDATE la ligne (ship_date, deliver_date...)
- **Mesures** : délai entre étapes, status actuel

❗ C'est le SEUL type de table de faits qu'on met à jour. Les trois autres types sont insert-only.

DDL : snapshot accumulant

fact_order_pipeline -- une ligne = une commande, mise à jour à chaque jalon

```
-- GRAIN : une ligne = une commande
-- UPDATE (exception) : on met à jour les colonnes NULL quand un jalon est atteint
CREATE OR REPLACE TABLE fact_order_pipeline AS
SELECT
    rp.pipeline_id,
    rp.order_id,
    CAST(rp.order_date AS DATE) AS order_date,
    CAST(rp.payment_date AS DATE) AS payment_date, -- NULL jusqu'au paiement
    CAST(rp.pick_date AS DATE) AS pick_date, -- NULL jusqu'au picking
    CAST(rp.ship_date AS DATE) AS ship_date, -- NULL jusqu'à l'expédition
    CAST(rp.delivery_date AS DATE) AS delivery_date, -- NULL jusqu'à la livraison
    rp.current_status,
    CAST(rp.days_order_to_deliver AS INTEGER) AS days_order_to_deliver,
    p.product_key,
    st.store_key,
    c.customer_key,
    -- Flags booléens par jalon atteint (utile pour le funnel)
    CASE WHEN rp.payment_date IS NOT NULL THEN TRUE ELSE FALSE END AS reached_payment,
    CASE WHEN rp.pick_date IS NOT NULL THEN TRUE ELSE FALSE END AS reached_pick,
    CASE WHEN rp.ship_date IS NOT NULL THEN TRUE ELSE FALSE END AS reached_ship,
    CASE WHEN rp.delivery_date IS NOT NULL THEN TRUE ELSE FALSE END AS reached_delivery
FROM raw_fact_order_pipeline rp
JOIN dim_product p ON p.product_id = rp.product_id
JOIN dim_store st ON st.store_id = rp.store_id
JOIN dim_customer c ON c.customer_id = rp.customer_id
                    AND CAST(rp.order_date AS DATE)
                    BETWEEN c.effective_from AND c.effective_to;

-- Requête funnel (exercice 3) :
-- SELECT current_status, COUNT(*) AS nb_commandes,
--        ROUND(AVG(days_order_to_deliver), 1) AS delai_moyen
```

04 Type 4 : Factless Fact

L'existence EST le fait

La table de faits sans mesure

Enregistrer qu'un événement ou une couverture a eu lieu

- **fact_promo_exposure** : customer_key, campaign_id, channel_key, exposure_date
- Pas de revenue, pas de quantité -- juste l'existence de la combinaison
- **Question** : "combien de clients ont été exposés à la campagne X ?"
- **La mesure implicite** : COUNT(*) = nombre d'expositions
- **Utile pour** : couverture, éligibilité, présence, inscription
- **L'inverse aussi est utile** : quels clients n'ont PAS été exposés ? (NOT EXISTS)

❗ La factless fact est un "registre de présence". Ce qui n'est PAS dans le registre est aussi une information précieuse.

DDL : factless fact table

fact_promo_exposure -- l'existence de la ligne est le fait

```
-- GRAIN : un client × une campagne × une date d'exposition
-- PAS de mesure numérique : la présence EST l'information
CREATE OR REPLACE TABLE fact_promo_exposure AS
SELECT
    re.exposure_id,
    CAST(re.exposure_date AS DATE) AS exposure_date,
    re.campaign_id,           -- dimension dégénérée (pas de table dim_campaign)
    c.customer_key,
    ch.channel_key
FROM raw_fact_promo_exposure re
JOIN dim_customer c
    ON c.customer_id = re.customer_id
   AND CAST(re.exposure_date AS DATE) BETWEEN c.effective_from AND c.effective_to
JOIN dim_channel ch ON ch.channel_id = re.channel_id;

-- Questions CEO avec COUNT(*) :
-- SELECT campaign_id, COUNT(*) AS nb_expositions
-- FROM fact_promo_exposure GROUP BY 1 ORDER BY 2 DESC;

-- Clients NON exposés (l'inverse) :
-- SELECT COUNT(*) FROM dim_customer
-- WHERE customer_key NOT IN (SELECT DISTINCT customer_key FROM fact_promo_exposure)
-- AND is_current = TRUE;
```

❗ Si vous ajoutez un montant (ex. coût d'impression par exposition), ce n'est plus une factless fact — c'est une table transactionnelle avec mesure.

L'arbre de décision

Quel type pour quel processus ?

- Le processus enregistre un événement ponctuel ? → Transaction
- Le processus capture un état periodique ? → Periodic Snapshot
- Le processus suit un cycle de vie avec des étapes ? → Accumulating Snapshot
- Le processus enregistre une presence/couverture sans mesure ? → Factless
- Cet arbre est le livrable le plus reutilisable de la session

Votre copilote cette semaine

Comment travailler avec votre assistant IA pour les types de faits

✓ PROMPTS QUI MARCHENT

- Explique les 4 types de tables de faits avec un exemple NexaMart chacun.
- Mon inventaire est un periodic snapshot. Quel grain ? Genere le DDL.
- Le pipeline commandes est un accumulating snapshot. Genere le DDL avec dates d'étape.

✗ PROMPTS À REJETER

- l'assistant confond periodic et accumulating snapshot
- l'assistant ajoute des mesures a une factless fact table

05 Exercice 1 : Table transactionnelle

Vérifiez le grain et écrivez deux requêtes CEO

Exercice 1 — Votre mission

Durée estimée : 35 min

- 1. make generate && make load — vérifiez que fact_orders_transaction est chargée
- 2. `SELECT COUNT(*), COUNT(DISTINCT transaction_id) FROM fact_orders_transaction`
- 3. **Répondez** : total = distinct ? Combien de transaction_type (sale/return/exchange) ?
- 4. **Requête CEO 1** : revenue total (SUM(amount)) par catégorie de produit
- 5. **Requête CEO 2** : top 5 magasins par revenue (SUM(amount))
- 6. Sauvegardez dans `sql/fact-types/s09-transaction.sql`

❗ Résultat attendu step 2 : `COUNT(*) = COUNT(DISTINCT transaction_id)` — le grain est la transaction (insert-only, une ligne par transaction).

Exercice 1 — Résultat attendu

Comparez votre output à ces chiffres indicatifs

```
-- Vérification du grain
SELECT COUNT(*) AS total_lignes,
       COUNT(DISTINCT transaction_id) AS nb_transactions
FROM fact_orders_transaction;
-- Attendu : total_lignes = nb_transactions (grain = transaction, une ligne chacune)

-- Revenue par catégorie (votre résultat variera selon vos données générées)
SELECT p.category,
       COUNT(*)      AS nb_transactions,
       SUM(f.quantity) AS unites,
       SUM(f.amount)  AS revenue_brut
FROM fact_orders_transaction f
JOIN dim_product p ON p.product_key = f.product_key
GROUP BY 1
ORDER BY revenue_brut DESC;
-- Attendu : 8-10 catégories, revenue additif (SUM est correct ici)
```

❗ fact_orders_transaction expose amount et quantity (pas de cost/line_total). Si la table n'existe pas, vérifiez que make load a bien exécuté sql/facts/. Consultez docs/TROUBLESHOOTING.md.

06

Exercice 2 : Snapshot periodique

Observez le piège de la semi-additivité

Exercice 2 — Votre mission

Durée estimée : 35 min

- 1. `SELECT COUNT(*) FROM fact_daily_inventory` -- attendu ~6 500 lignes
- 2. **Vérifiez le grain** : `SELECT DISTINCT product_key, store_key, snapshot_date` -- même COUNT ?
- 3. Exécutez la requête SUM (fausse) et la requête AVG (correcte) de la diapositive précédente
- 4. **Notez l'écart dans votre brief** : "SUM donne X, AVG donne Y, l'écart est Z×"
- 5. **Requête CEO** : stock moyen des 5 derniers jours disponibles, par magasin
- 6. Sauvegardez dans `sql/fact-types/s09-periodic-snapshot.sql`

❗ Le piège est intentionnel. Vous devez OBSERVER le chiffre absurde produit par SUM avant d'expliquer pourquoi il est faux dans votre brief.

07

Exercice 3 : Snapshot accumulant

Analysez le pipeline de commandes NexaMart

Exercice 3 — Votre mission

Durée estimée : 35 min

- 1. `SELECT COUNT(*), COUNT(DISTINCT order_id) FROM fact_order_pipeline`
- 2. **Requête funnel** : `GROUP BY current_status, COUNT(*), AVG(days_order_to_deliver)`
- 3. **Comptez les NULLs par colonne de jalon** : combien de commandes n'ont pas atteint `delivery_date` ?
- 4. **Requête CEO** : quel % des commandes est livré ? Quel est le délai moyen de livraison ?
- 5. **Expliquez dans le brief** : pourquoi les `delivery_date` NULL ne sont pas une erreur
- 6. Sauvegardez dans `sql/fact-types/s09-accumulating.sql`

❗ Un NULL dans `delivery_date` signifie que la commande est en cours, pas que les données sont corrompues. Votre brief doit expliquer cela au CEO.

Exercice 3 — Requêtes de référence

Pipeline funnel et analyse des délais

```
-- Funnel par statut
SELECT current_status,
       COUNT(*) AS nb_commandes,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER ()), 1) AS pct_total,
       ROUND(AVG(days_order_to_deliver), 1) AS delai_moyen_jours
FROM fact_order_pipeline
GROUP BY 1
ORDER BY nb_commandes DESC;

-- Taux de complétion par jalon
SELECT
  ROUND(100.0 * SUM(reached_payment::INT) / COUNT(*), 1) AS pct_payment,
  ROUND(100.0 * SUM(reached_pick::INT) / COUNT(*), 1) AS pct_pick,
  ROUND(100.0 * SUM(reached_ship::INT) / COUNT(*), 1) AS pct_ship,
  ROUND(100.0 * SUM(reached_delivery::INT) / COUNT(*), 1) AS pct_delivery
FROM fact_order_pipeline;

-- Attendu : pct_payment > pct_pick > pct_ship > pct_delivery
```

❗ Si pct_delivery = 100%, vérifiez que vos données comportent bien des commandes "en cours" (delivery_date IS NULL). Consultez docs/TROUBLESHOOTING.md.

08

Exercice 4 : Factless fact table

Couverture promotionnelle et requête inverse

Exercice 4 — Votre mission

Durée estimée : 30 min

- 1. `SELECT COUNT(*), COUNT(DISTINCT customer_key) FROM fact_promo_exposure`
- 2. **Requête couverture** : combien d'expositions par `campaign_id` ?
- 3. **Requête canal** : quel canal a généré le plus d'expositions ?
- 4. **Requête inverse** : combien de clients actifs n'ont PAS été exposés à une campagne ?
- 5. **Justifiez dans le brief** : pourquoi il n'y a pas de montant dans cette table
- 6. Sauvegardez dans `sql/fact-types/s09-factless.sql`

❗ La requête inverse (`NOT IN` / `NOT EXISTS`) est aussi importante que la requête directe. Le CEO veut savoir QUI n'a pas été atteint, pas seulement QUI l'a été.

09 Arbre de décision et brief exécutif

Le livrable le plus réutilisable de la session

Livrable final

Carte des processus NexaMart + brief exécutif

- **docs/fact-type-decision-tree.md** : le template est déjà dans votre repo
- **Complétez le tableau** : 4 processus NexaMart, le type correct, la justification
- **Brief CEO** : répondre avec des chiffres issus de vos 4 tables de faits
- **Un chiffre par table** : revenue total, stock moyen, % livré, nb expositions
- L'arbre de décision doit être applicable à de nouveaux processus non NexaMart

Soumission

⚠ Echéance : avant le 15 juin 18h00

- ⚠ La séance S10 (15 juin) est le Comité du board — non noté
- **Fichiers** : answers/S09_executive_brief.md + docs/fact-type-decision-tree.md
- **SQL** : sql/fact-types/s09-transaction.sql, s09-periodic-snapshot.sql, s09-accumulating.sql, s09-factless.sql
- **Evaluation** : modèle 40%, validation 25%, justification 20%, traçabilité 10%, reproductibilité 5%
- **Commitez** : git add -A && git commit -m "S09: 4 types de tables de faits" && git push
- Consultez docs/verify-before-pushing.md avant de pousser

Soumission (suite)

- Vidéo de synthèse disponible dans docs/lab-guides/GIS805-09_lab.md section 0

Ou en sommes-nous ?

Votre parcours en 12 séances

- S01-S08 : Étoile, grain, SCD, dims, intégration, ponts [FAIT]
- >> **S09** : 4 types de tables de faits [CETTE SEMAINE — AUTONOME]
- S10 : Comité du board (non noté) — 15 juin
- S11 : Handoff pack + brief board (10%) — 18 juin
- S12 : Examen final — 22 juin

A retenir cette semaine

- 1 Transaction = événement ponctuel. Snapshot = état a un moment. Accumulating = cycle de vie.
- 2 Factless = l'existence est le fait. Pas de mesure, mais une reponse réelle.
- 3 Le snapshot periodique est semi-additif -- ne SUM jamais le stock sur le temps.
- 4 L'accumulating snapshot est le SEUL type de table de faits qu'on UPDATE.
- 5 Remise avant le 15 juin 18h00 -- S10 est le Comité du board (non noté).